

Lab 3: CoreMark Benchmark Report

Name: Ali Ayhan Gunder 2021400219

Partner: Rojhat Demirbas 2022400057

FF34G14

November 6, 2025

Contents

1	System Information	2
2	Benchmark Results	2
	2.1 Default Build Performance	2
	2.2 Compiler Optimization Comparison	2
3	Analysis of Results	2
	3.1 Performance Observations	2
4	CoreMark Algorithm Explanation	4
	4.1 List Processing (Linked Lists)	4
	4.2 Matrix Manipulation	4
	4.3 State Machine	4
	4.4 CRC (Cyclic Redundancy Check)	4
5	RISC-V ISA Extension Sets	5
	5.1 RV32I — Base Integer Instruction Set (32-bit)	5
	5.2 RV32IM — Base + Multiply/Divide Extension	5
	5.3 RV32IMC — Base + Multiply/Divide + Compressed Instructions	5
	5.4 RV32GC — General-Purpose Configuration	5
	5.5 RV64I — Base Integer Instruction Set (64-bit)	5
	5.6 Comparison: Why Different Extension Sets Matter	5
6	Conclusions	6
7	References	6
8	Appendix	6
	8.1 Sample CoreMark Output	6
	8.2 Build Commands Used	7
	8.3 CPU Information Output	7

1 System Information

CPU Model:	11th Gen Intel(R) Core(TM) i7-11375H @ 3.30GHz
Base Clock Frequency:	3300 MHz
Max Clock Frequency:	3302 MHz
Number of Cores:	4
Number of Logical Processors:	8
Operating System:	Windows
Compiler:	GCC version 15.2.0 (MSYS2 MinGW-w64)

2 Benchmark Results

2.1 Default Build Performance

Run #	Iterations/Sec
1	19290.12
2	19227.07
3	19342.36
4	19753.74
5	19490.64
Average	19420.79

2.2 Compiler Optimization Comparison

Optimization Flag	Run 1	Run 2	Run 3	Run 4	Run 5	Average	% Change vs Default
-O0	4607.23	4617.87	4605.82	4631.06	4926.11	4677.62	-75.91%
-O1	18389.11	18310.55	18392.50	18426.39	18374.47	18378.60	-5.37%
-O2 (Default)	19290.12	19227.07	19342.36	19753.74	19490.64	19420.79	0%
-O3	20458.27	20578.95	20529.67	20604.40	20567.67	20547.79	+5.80%
-Ofast	17572.63	19228.30	20207.46	20326.58	20348.64	19536.72	+0.60%
-march=native	19021.05	19146.08	19211.07	19133.87	19196.31	19141.68	-1.44%

Formula for % Change:

$$\text{Percent Change} = \frac{(\text{New Value} - \text{Default Value})}{\text{Default Value}} \times 100$$

3 Analysis of Results

3.1 Performance Observations

Impact of Optimization Levels The benchmark results clearly demonstrate the significant impact of compiler optimization on CPU performance. The progression from -O0 (no optimization) to -Ofast shows dramatic performance improvements:

- **-O0 (No Optimization):** At 4,678 iterations/sec, this represents a 75.91% performance **decrease** compared to the default -O2 setting. Without optimization, the compiler generates straightforward but inefficient machine code, resulting in excessive memory operations, redundant calculations, and poor instruction scheduling.

- **-O1 (Basic Optimization):** Performance jumps to 18,379 iterations/sec, recovering most of the ground but still 5.37% slower than -O2. This level enables fundamental optimizations like common subexpression elimination and dead code removal without aggressive transformations.
- **-O2 (Default):** At 19,421 iterations/sec, this is GCC’s recommended optimization level for production code, balancing performance with compilation time and code size. It includes loop optimizations, vectorization, and instruction scheduling.
- **-O3 (Aggressive Optimization):** Shows a 5.80% improvement over -O2 (20,548 iterations/sec), enabling more aggressive loop unrolling, function inlining, and other transformations that may increase code size but improve execution speed. This achieved the best overall performance among all tested configurations.
- **-Ofast (Maximum Speed):** Achieves 19,537 iterations/sec (+0.60% over -O2). While this level relaxes strict standards compliance (e.g., IEEE 754 floating-point) for additional speed, in this benchmark it only marginally outperformed -O2, showing that the aggressive optimizations don’t always translate to significant gains depending on the workload.

The key finding is that going from -O0 to -O2 provides a **315% performance increase**, while -O2 to -O3 adds another 5.80%. This demonstrates that most performance gains come from basic optimizations, with diminishing returns at higher levels.

Performance Variations Across all optimization levels, results showed reasonable consistency between runs, with variations typically under 5%. Some notable observations:

- The -O0 runs showed moderate variation (from 4,605 to 4,926 iterations/sec), likely due to longer execution times making system variability more apparent.
- Higher optimization levels (-O2, -O3) showed relatively tight clustering of results, indicating more predictable performance.
- The -Ofast configuration showed higher variance, with Run 1 at 17,573 significantly lower than Runs 3–5 (20,207–20,349), suggesting initial cold-start effects or CPU ramping behavior.

The variations can be attributed to factors such as CPU thermal throttling, turbo boost behavior, background processes, and operating system scheduling decisions. Running multiple iterations and calculating averages helps mitigate these effects.

Architecture-Specific Optimizations The `-march=native` flag, which tells the compiler to optimize for the specific CPU architecture (11th Gen Intel Core i7-11375H), surprisingly performed **worse** than -O2 by 1.44%, achieving only 19,142 iterations/sec. This unexpected result can be explained by several factors:

- **Instruction Mix Trade-offs:** While `-march=native` enables advanced instructions specific to the Tiger Lake architecture (AVX-512, enhanced SSE), CoreMark’s workload may not benefit from these specialized instructions. The benchmark focuses on integer operations, pointer chasing, and branching rather than vectorizable floating-point operations.

- **Code Size Impact:** Architecture-specific optimizations can increase code size, potentially leading to more instruction cache misses, which would harm performance on CoreMark’s relatively small but frequently executed loops.
- **Interaction with Other Flags:** When `-march=native` is used without other optimization flags, it defaults to `-O0`, which may explain lower performance in some configurations if not combined properly.

This result highlights an important lesson for embedded system development: **blindly enabling all optimizations doesn’t guarantee better performance**. The effectiveness of optimizations depends heavily on the workload characteristics, and benchmark-driven validation is essential when selecting processor designs and compiler settings for production systems.

4 CoreMark Algorithm Explanation

4.1 List Processing (Linked Lists)

- **Operations:** Finding/removing elements, sorting, traversal
- **Tests:** Pointer chasing, branch prediction, cache locality
- **Relevance:** Task scheduling, memory management

4.2 Matrix Manipulation

- **Operations:** Matrix multiplication, addition, integer operations
- **Tests:** Computational throughput, data locality
- **Relevance:** Signal processing, graphics, control systems

4.3 State Machine

- **Operations:** State transitions, string parsing, pattern matching
- **Tests:** Branch prediction, control flow
- **Relevance:** Protocol handlers, parsers, control logic in embedded systems

4.4 CRC (Cyclic Redundancy Check)

- **Operations:** Bit manipulation, checksum calculation
- **Tests:** Bit-level operations and integer arithmetic
- **Relevance:** Data integrity verification, communications

CoreMark Characteristics: No I/O operations; minimal memory footprint; deterministic; portable; self-validating.

5 RISC-V ISA Extension Sets

5.1 RV32I — Base Integer Instruction Set (32-bit)

Foundation ISA with 32-bit registers, basic arithmetic, load/store, and control flow. Ideal for simple embedded systems.

5.2 RV32IM — Base + Multiply/Divide Extension

Adds hardware multiply/divide (M extension): MUL, DIV, REM instructions. Major performance boost for arithmetic workloads.

5.3 RV32IMC — Base + Multiply/Divide + Compressed Instructions

Adds compressed (C) 16-bit instructions. Reduces code size 25–30%, improves cache utilization, no performance penalty.

5.4 RV32GC — General-Purpose Configuration

Combines IMAFD + C: integer, atomic, and floating-point support. Suitable for advanced embedded and application processors.

5.5 RV64I — Base Integer Instruction Set (64-bit)

Extends address space and registers to 64 bits. Adds LD/SD instructions and 32-bit word operations. Ideal for large-memory or high-performance systems.

5.6 Comparison: Why Different Extension Sets Matter

Feature	RV32I	RV32IM	RV32IMC	RV32GC	RV64I
Multiply/Divide (or +M)	Software	Hardware	Hardware	Hardware	Software
Code Size	Baseline	Baseline	70–75%	70–75%	Larger
Floating-Point (or +FD)	Software	Software	Software	Hardware	Software
Atomics (+A)	No	No	No	Yes	Optional
Address Space	32-bit	32-bit	32-bit	32-bit	64-bit
Typical Use	Educational	Embedded	Memory-		
limited	General-				
purpose	High-				
performance					

6 Conclusions

1. **Compiler Optimizations are Critical:** The 315% performance difference between unoptimized (`-O0`) and default optimized code (`-O2`) underscores that compiler optimization is not optional for performance-critical embedded applications. The results show that `-O3` provided the best performance with a 5.80% gain over `-O2`, while `-Ofast` offered minimal improvement, suggesting that `-O2` or `-O3` represent the sweet spot for most production code.
2. **Benchmarking Enables Informed Decisions:** Understanding that a given workload achieves $\sim 20,548$ iterations/sec on an Intel Core i7-11375H with GCC 15.2.0 provides a baseline for comparing RISC-V and other processor architectures in future labs.
3. **Algorithm Understanding Matters:** CoreMark’s focus on four key computational patterns (list processing, matrix operations, state machines, and CRC) provides a representative workload for embedded systems. Recognizing which algorithms stress which processor features helps match processor capabilities to application requirements.
4. **RISC-V Configurability Trades Off Complexity vs. Performance:** Selecting the right extension set (e.g., RV32IMC for code density vs. RV32GC for floating-point capability) directly impacts chip cost, power consumption, and application performance.
5. **Validate Architecture-Specific Optimizations:** The counterintuitive result that `-march=native` performed worse than generic `-O2` optimization (by 1.44%) shows assumptions about performance must be tested.

7 References

1. CoreMark Official Repository: <https://github.com/eembc/coremark>
2. EEMBC CoreMark Documentation: <https://www.eembc.org/coremark/>

8 Appendix

8.1 Sample CoreMark Output

2K performance run parameters for coremark.

```
CoreMark Size      : 666
Total ticks        : 14538
Total time (secs): 14.538000
Iterations/Sec     : 20635.575733
Iterations         : 300000
Compiler version   : GCC15.2.0
Compiler flags     : -O2
Memory location    : STACK
seedcrc            : 0xe9f5
```

```

[0]crclist      : 0xe714
[0]crcmatrix    : 0x1fd7
[0]crcstate     : 0x8e3a
[0]crcfinal     : 0xcc42
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 20635.575733 / GCC15.2.0 -O2 / STACK

```

8.2 Build Commands Used

```

# Set up PATH for gcc and make (if needed in new terminal)
$env:Path = "C:\Program Files\Git\bin;C:\msys64\mingw64\bin;" + $env:Path

# Navigate to CoreMark directory
cd c:\Users\gunde\Desktop\344Labs\344LAB3\coremark

# Build with different optimization levels
mingw32-make PORT_DIR=simple XCFLAGS="-O0" compile
mingw32-make PORT_DIR=simple XCFLAGS="-O1" compile
mingw32-make PORT_DIR=simple XCFLAGS="-O2" compile
mingw32-make PORT_DIR=simple XCFLAGS="-O3" compile
mingw32-make PORT_DIR=simple XCFLAGS="-Ofast" compile
mingw32-make PORT_DIR=simple XCFLAGS="-march=native" compile

# Run benchmark (example for -O2)
.\coremark.exe 0x0 0x0 0x66 0 7 1 2000

# Clean before rebuilding
mingw32-make PORT_DIR=simple clean

```

8.3 CPU Information Output

```

PS C:\Users\gunde\Desktop\344Labs\344LAB3> Get-WmiObject Win32_Processor | Select-Obj

```

Name	: 11th Gen Intel(R) Core(TM) i7-11375H @ 3.30GHz
MaxClockSpeed	: 3302
NumberOfCores	: 4
NumberOfLogicalProcessors	: 8